

Tema 03. Redes de Neuronas

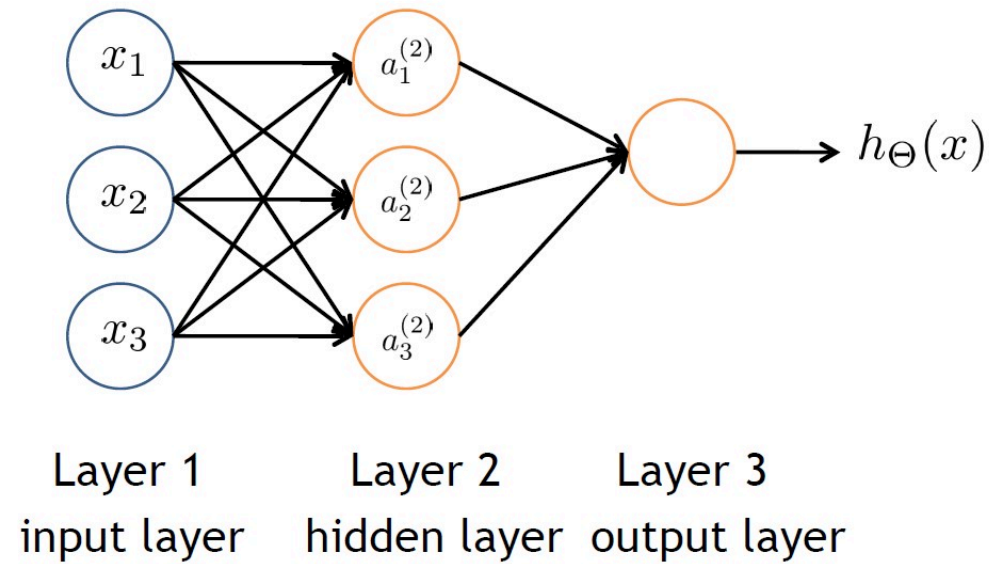
Autor: Ismael Sagredo Olivenza

3.4 ¿Cómo aprende una RNA?

Recuerda la propagación hacia delante:

$$a_j^l = g\left(\left(\sum_{i=1}^{|a^{l-1}|} \Theta_{j,i}^j a_i^{l-1}\right) + b_j^l\right) \forall j \in [1..|a^l|], l \in [1..|layers|]$$

Donde **layers** son las capas del modelo, $|layers|$ número de capas, $|a^l|$ número de neuronas en la capa l , $|a^{l-1}|$ número de neuronas en la capa anterior y g la función de activación (ReLU, sigmoidal, etc).



$$a_1^2 = g(\Theta_{1,1}^1 x_1 + \Theta_{1,2}^1 x_2 + \Theta_{1,3}^1 x_3 + b_1^1)$$

$$a_2^2 = g(\Theta_{2,1}^1 x_1 + \Theta_{2,2}^1 x_2 + \Theta_{2,3}^1 x_3 + b_2^1)$$

$$a_3^2 = g(\Theta_{3,1}^1 x_1 + \Theta_{3,2}^1 x_2 + \Theta_{3,3}^1 x_3 + b_3^1)$$

$$y' = h_{\Theta}(x) = a_1^3 = g(\Theta_{1,0}^2 + \Theta_{1,1}^2 a_1^2 + \Theta_{1,2}^2 a_2^2 + \Theta_{1,3}^2 a_3^2)$$

Como en los algoritmos anteriores, necesitamos la función de coste J . Partimos de la función de coste de regresión logística:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(y'_i) + (1 - y_i) \log(1 - y'_i)]$$

Pero podemos tener más de una salida y'_i = i-ésima salida

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K [y_{k,i} \log(y'_{k,i}) + (1 - y_{k,i}) \log(1 - y'_{k,i})]$$

La suma del error de todas las K neuronas de salida es el error producido por un ejemplo, (error medio - la media de los m ejemplos)

3.4.1 ¿Cómo modificar los pesos de la RNA para aprender?

El descenso de gradiente es tu amigo :), en regresión logística:

$$\frac{\partial J(w, b)}{\partial w_j} = \sum_{i=1}^m y'_i - y_i x_{j,i}$$

En NN:

$$\frac{\partial J(\Theta)}{\partial \Theta_{j,i}^l} = \sum_{k=1}^m \delta_j^{l+1}(k) a_i^l(k)$$

La derivada parcial de un peso ($\Theta_{j,i}^l$) que conecta la neurona j de la capa $l+1$ con la neurona i de la capa l de la matriz de pesos Θ^l = al error producido en la capa $l+1$ (que llamamos delta δ), por el ejemplo k , por la activación de la neurona i de la capa l por el ejemplo k .

Ahora tenemos que determinar cuál es el error. En la capa de salida ($l = L$ o última capa) el error será la diferencia entre la salida esperada y la salida real.

$$\delta_j^L = (y'_j - y_j)$$

Otras implmentaciones derivan la salida **pero nosotros no:**

$$\delta_j^L = (y'_j - y_j)g'(y'_j) = (a_j^L - y_j)g'(a_j^L)$$

En la penúltima capa (L-1), sin embargo, debemos propagar el error a través de la red utilizando los pesos.

$$\delta_i^{L-1} = \Theta_{j,i}^{L-1} \delta_j^L g'(a_i^{L-1})$$

Es decir: el error de la capa siguiente X por el peso que conecta la neurona de la que calculamos su delta, por la derivada la neurona donde calculamos delta (g'), que en el caso de la función logística es:

$$g'(a_i^l) = a_i^l(1 - a_i^l)$$

En otras palabras, la derivada de la función consiste en multiplicar la activación obtenida por la neurona, por 1 menos la activación obtenida por la neurona.

Generalizando para cualquier capa, el factor delta del error puede expresarse del siguiente modo:

$$\delta_i^l = \Theta_{j,i}^l \delta_j^{l+1} g'(a_i^l)$$

Por lo tanto, la disminución del gradiente para cualquier peso en cualquier capa puede expresarse como: $\Theta_{j,i}^l = \Theta_{j,i}^l - \alpha \frac{\partial J(\Theta)}{\partial \Theta_{j,i}^l}$ donde:

$$\frac{\partial J(\Theta)}{\partial \Theta_{j,i}^l} = \frac{1}{m} \sum_{k=1}^m \delta_j^{l+1}(k) a_i^l(k)$$

Y los deltas del resto de capas se calcularían del siguiente modo:

Regla Delta Generalizada Para todas las capas desde la 2 hasta la L-1 (las capas ocultas) para el ejemplo k-ésimo (k) de entrenamiento:

$$\delta_i^l(k) = \sum_{j=1}^{size(a^{l+1})} (\Theta_{j,i}^l \cdot \delta_j^{l+1}(k) \cdot g'(a_i^l(k)))$$

Donde la derivada será la siguiente:

$$g'(a_i^l(k)) = a_i^l(k) * (1 - a_i^l(k))$$

Para todos los k ejemplos de entrenamiento, y para cada una de las i neuronas de la capa l correspondiente. (Un delta por cada neurona)

Resumen de ecuaciones del backpropagation:

Reglas delta:

Para la ultima capa:

$$\delta_j^L(k) = y_j'(k) - y_j(k)$$

Para el resto de capas ocultas:

$$\delta_i^l(k) = \sum_{j=1}^{size(a^{l+1})} (\Theta_{j,i}^l \delta_j^{l+1}(k) g'(a_i^l(k)))$$

Gradientes:

Para cada peso i y j de la matriz Θ^l

$$\frac{\partial J(\Theta)}{\partial \Theta_{j,i}^l} = \frac{1}{m} \sum_{k=1}^m \delta_j^{l+1}(k) \cdot a_i^l(k)$$

Donde K es el ejemplo correspondiente, $a_i^l(k)$ es la activación de la neurona i de la capa l para el ejemplo k

Actualización de los pesos:

$$\Theta_{j,i}^l = \Theta_{j,i}^l - \alpha \frac{\partial J(\Theta)}{\partial \Theta_{j,i}^l}$$

3.4.2 Algoritmo:

- Inicializar los pesos y umbrales de la red con valores aleatorios.
- Para cada patrón de entrada
 - Cargar las entradas en las neuronas de entrada.
 - Propagar la entrada a través de la red (feedforward).
 - Calcular el error de ese patrón
 - Calcular todos los δ para todas las neuronas de la red.
 - Calcular los **gradientes** para todas las conexiones de la red
 - Almacenar el error cometido
- Actualizar los pesos y calcular el error medio

3.4.3 Regularization

Aplicamos la regularización L2, al coste hay que sumarle la suma de todos los pesos de la red al cuadrado **NOTA: (El sesgo o bias no debe regularizarse, por lo que hay que sacarlo del computo)**

$$JL2(\Theta) = J(\Theta) + \lambda \frac{1}{2m} \sum_{l=1}^{|L|-1} \sum_{j=1}^{Dim(l+1)} \sum_{i=1}^{Dim(l)} (\Theta_{j,i}^l)^2$$

El término gradiente en la regularización (derivada del anterior):

$$\partial JL2(\Theta) = \partial J(\Theta) + \lambda \frac{1}{m} \sum_{l=1}^{|L|-1} \sum_{j=1}^{Dim(l+1)} \sum_{i=1}^{Dim(l)} (\Theta_{j,i}^l)$$

Explicación

El **coste** regularizado es sumarle al coste que ya había, la suma al cuadrado de todos los pesos de la red (sin contar los sesgos) multiplicado por el factor de regularización λ entre $2m$.

El **gradiente regularizado** es igual pero dividido entre m y sin elevar al cuadrado los pesos.

3.4.4 Ejemplo

Neuronas de entrada 2

Neuronas capa oculta 2

Neuronas de salida 2

Salida esperada 0.7,0.1

Pesos de las matrices Tetha1,Tetha2:

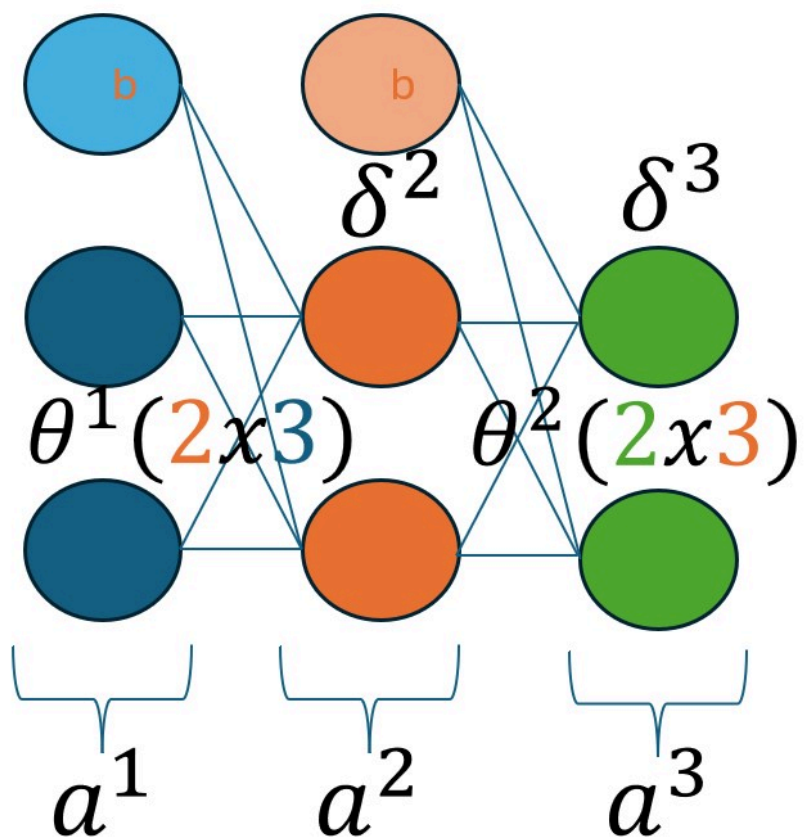
Tetha1 = 2X2 $\begin{bmatrix} 0.1 & 0.2 \\ 0.2 & 0.1 \end{bmatrix}$ obviamos b.

Tetha2 = 2X2 $\begin{bmatrix} 0.2 & 0.3 \\ 0.1 & 0.1 \end{bmatrix}$ obviamos b.

Salida = $a^3 = (0.5, 0.4)$

$a^2 = (0.25, 0.5)$

Calculamos el backpropagation:



$$\theta^1(2 \times 3)$$

θ			
1	0.1	0.1	0.2
2	0.5	0.2	0.1
	0	1	2

$$\theta^2(2 \times 3)$$

θ			
1	0.1	0.2	0.3
2	0.5	0.1	0.1
	0	1	2

Calculamos: $\delta_j^L = (y'_j - y_j)$ donde $L=3$ y $j=\{1,2\}$

$$\delta_1^3 = (0.5 - 0.7) = -0.2 \Rightarrow \delta_i^L = (y'_i - y_i) \text{ donde } L=3 \text{ y } j=\{1,2\}$$

$$\delta_2^3 = (0.4 - 0.1) = 0.3$$

$$\delta_1^2 = \Theta^2[1, 1] * \delta_1^3 * g'(a_1^2) + \Theta^2[2, 1] * \delta_2^3 * g'(a_1^2)$$

$$\delta_1^2 = 0.2 * (-0.2) * [0.25 * (1 - 0.25)] + 0.1 * 0.3 * [0.25 * (1 - 0.25)]$$

$$\delta_2^2 = \Theta^2[1, 2] * \delta_1^3 * g'(a_2^2) + \Theta^2[2, 2] * \delta_2^3 * g'(a_2^2)$$

$$\delta_2^2 = 0.3 * (-0.2) * [0.5 * (1 - 0.5)] + 0.1 * 0.3 * [0.5 * (1 - 0.5)]$$

$$\Theta^1 = \Theta^1 - \alpha * \delta^2(a^1)$$

$$\Theta^2 = \Theta^2 - \alpha * \delta^3(a^2)$$

¿Por qué no inicializamos a cero?

Porque $\Theta_{01}^1 = \Theta_{02}^1$, la activación de la capa oculta $a_1^2 = a_2^2$, por lo tanto las derivadas serán iguales. Las neuronas ocultas computan la misma combinación de inputs y por tanto **la misma característica oculta**.

Inicialización aleatoria

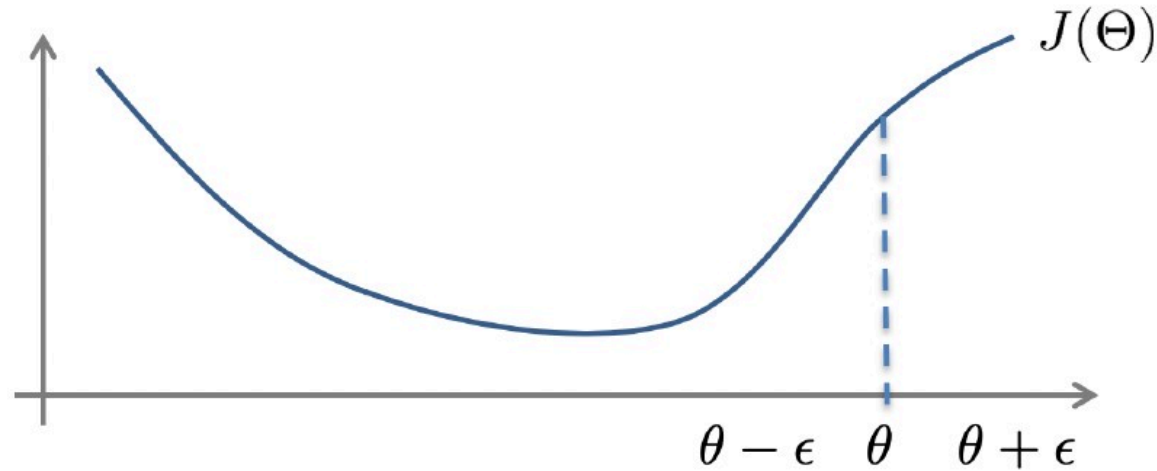
inicializar para cada $\Theta_{i,j}^l$ a un valor aleatorio entre $[-\varepsilon, \varepsilon]$

```
theta1 = np.random.uniform(low=-EPSILON, high=EPSILON, size=theta1.shape())  
theta2 = np.random.uniform(low=-EPSILON, high=EPSILON, size=theta2.shape())
```

3.5 Algoritmos de optimización

- **Descenso por gradiente:** explicado anteriormente.
- **BFGS / L-BFGS:** utiliza una cantidad limitada de memoria del ordenador. Es un algoritmo popular para la estimación de parámetros en el aprendizaje automático. El problema objetivo del algoritmo es minimizar sobre valores no restringidos del vector R
- **ADAM:** optimización basada en el gradiente de funciones objetivo estocásticas. Se utiliza en problemas que son grandes en términos de datos y parámetros o con gradientes muy ruidosos y dispersos ([paper](#))
- **Stochastic gradient descent:** estima el gradiente utilizando un subconjunto de los datos. Se utiliza en problemas de alta dimensión

Numerical estimation of gradients



$$\frac{d}{d\Theta} J(\Theta) = \frac{J(\Theta + \epsilon) - J(\Theta - \epsilon)}{2\epsilon} \quad \epsilon = 10^{-4}$$

Implement:

```
gradApprox = ( J(theta + EPSILON) -  
                J(theta - EPSILON) ) / ( 2*EPSILON )
```

Podemos utilizarlo para comprobar que los cálculos se realizan correctamente. El valor de la estimación numérica debe ser similar al valor obtenido mediante el descenso por gradiente.

```
for i in range(len(thetaVec)):
    thetaPlus = thetaVec
    thetaPlus[i] = thetaPlus[i] + EPSILON
    thetaMinus = thetaVec
    thetaMinus[i] = thetaMinus[i] - EPSILON
    gradApprox[i] = (J(thetaPlus) - J(thetaMinus))
                    /(2*EPSILON)
```